Understanding logrotate on Ubuntu part 2

In this second part of the logrotate series we look at how to set up rotation for virtual host logs, as well as some troubleshooting techniques.

Applying knowledge

In the previous article we talked about what logrotate does and how you can configure it. In this article we'll apply this new knowledge to putting together a log rotation solution for a custom virtual host or two (or three, or four, etc.). We'll also look at some options for testing and troubleshooting logrotate.

Tying it all together: virtual host logs

To show how you can use logrotate for your own applications, let's look at an example that will come in handy for a lot of people: rotating logs for your custom virtual hosts. We'll use apache for this example, but it can be tweaked pretty easily for other web servers like nginx or lighttpd, usually just by changing the postrotate script.

First we'll want to create a file to hold the configuration that will tell logrotate what to do with the virtual host's log files. We won't edit the main config file or the web server's config file, since there's always a possibility that a future package upgrade might want to overwrite the config. Instead we'll make our own. Let's call it:

```
/etc/logrotate.d/virtualhosts
```

This example tosses all the virtual hosts into one file, but if you have one that's busier than others you may want to create separate config files to handle the needs of your different domains. We'll also specify several items that are probably already set in your main config, just so we cover all the bases.

The files

We'll say that we have two virtual domains, domain1.com and domain2.com, and that the log files for each are in /home/demo/public_html/(domain name)/log. The first thing we'll do in our config file is tell logrotate where to find the log files, then start the config block for them:

```
/home/demo/public_html/domain1.com/log/*log /home/demo/public_html/domain2.com/log/*log {
```

If you have more log directories or files to add, just insert them into that list.

Rotate

Next we'll want to make sure logrotate only keeps as many old logs as we want:

```
rotate 14
```

We'll use 14 in this example to keep two weeks' worth of logs, but you can of course adjust that number to something suitable to your requirements.

Interval

Now we'll tell the web server to rotate these logs daily (again, change it if you prefer a longer interval):

```
daily
```

Size (optional)

If you prefer a weekly rotation it's wise to specify a max log size as well, to be on the safe side. The max size setting doesn't make much difference if you have the logs rotating daily, but if you use "weekly" or longer instead you might also include the line:

```
size 50M
```

That way if a log starts getting too large (from unexpectedly heavy traffic, for example) it will be rotated early rather than allowing it to get too unwieldy.

Compression

We'll specify whether or not we want these logs to be compressed when they're archived. For this example we'll use delaycompress to account for the graceful restart of apache, which means we also need to turn compression on:

```
compress
delaycompress
```

Sharedscripts

You might have several virtual hosts, and that would mean several logs to rotate. To make sure the web server only gets restarted after all the rotations are done we add:

```
sharedscripts
```

Postrotate

We'll specify a postrotate script that will restart the web server:

```
postrotate
        /usr/sbin/apache2ctl graceful > /dev/null
endscript
```

And finally, we close the config block with a curly bracket:

```
}
```

The whole shebang

Once we bring it all together our config file will look like this:

```
/home/demo/public_html/domain1.com/log/*log /home/demo/public_html/domain2.com/log/*log {
        rotate 14
        daily
        compress
        delaycompress
        sharedscripts
        postrotate
                /usr/sbin/apache2ctl graceful > /dev/null
        endscript
```

```
}
```

You'll want to test that, of course, either by making sure you're watching things when the nightly cron jobs are run, or by running logrotate right now:

```
/usr/sbin/logrotate /etc/logrotate.conf
```

If you don't get any errors back you should be okay. But if you want to be absolutely certain you can run through some of the tests we would use when we suspect something isn't working right.

Testing logrotate

If you suspect logrotate is having some trouble, or you just want to make sure a new config you've put in place will work, there are some useful flags you can pass to logrotate when you run it from the command line:

Verbose

The verbose flag, "-v", tells logrotate to say what it's doing while it's doing it. It's very useful when trying to find out why logrotate doesn't rotate a log when you want it to.

Debug

The debug flag, "-d", tells logrotate to go through the motions of rotating logs but not *actually* rotate them. It can be handy if you want to test a new config file but don't want any actual log rotation run when you do (if you're working on a production server, for example).

The debug flag is good for checking that the config file is formatted properly and that logrotate can find the log files it would rotate. However, since it doesn't actually run the rotations it doesn't test some parts of the process like the postrotate scripts.

Force

The force flag, "-f", forces logrotate to rotate all logs when it runs, whether or not they would normally need to be rotated at that time. If you want to thoroughly test logrotate's configs this is the flag to use. Just remember that logrotate will be rotating logs and deleting old ones according to the configuration you've set up, so don't accidentally rotate out a recent log you needed to keep.

The force flag can be useful if you're convinced that logrotate should be rotating a log, but it isn't. Forcing the issue will help you tell if the problem is that logrotate doesn't think the log needed rotating (if you run with the force flag and the log is rotated), or if the problem is that logrotate isn't able to affect the log file (if you run it and nothing happens to the log).

Note that if logrotate is set to add a date to the name of an archived log, not even using the force flag will get logrotate to make a new archive in the same day (since the name it would use for the archive is already taken). In that circumstance you may need to rename the most recent archive (for each log file in a given config block) before you can force a log rotation.

Combining flags

The testing flags can be used together quite effectively. To have logrotate tell you what it would do if you made it rotate everything, but not *actually* rotate anything, you can

combine all three:

```
/usr/sbin/logrotate -vdf /etc/logrotate.conf
```

You'll get treated to a long list of things logrotate would do, including which log files it would rotate and what it would do during that process.

If you then want to test all the rotate configs in their entirety — including the scripts run after rotations — you can run logrotate without the debug flag:

```
/usr/sbin/logrotate -vf /etc/logrotate.conf
```

All the logs will be rotated, and skimming the output should help you catch any obvious problems. You'll also want to make sure that all your services are still running okay (that there was nothing wrong with the postrotate scripts), and that all the logs actually did get rotated.

How logrotate remembers

If you find that a log isn't rotating even though it's old enough that it should, a simple approach to fixing the problem is to manually run logrotate with the "-f" flag. But if you're the sort who wants to know why something's gone wrong there's one more file you can check before forcing a rotation:

```
/var/lib/logrotate/status
```

That file is where logrotate stores information about when it last rotated each log file. If you look inside that file you'll see something like:

```
logrotate state -- version 2
"/var/log/acpid.log" 2010-6-18
"/var/log/iptables.log" 2010-6-18
"/var/log/uucp.log" 2010-6-29
...
```

It's a straightforward format - the log file location is on the left, and the date when it was last rotated is on the right. Sometimes it can happen that the dates on your server get a little wonky (if you were tinkering with an NTP service or the like), and the date when a log was last rotated winds up being a future date. If that's happened you'll see it here.

If you want to check logrotate out with a particular log file but don't want to force everything to rotate, you can delete the log's entry from the logrotate status file. Then when you run logrotate normally, it should create a new entry for the log with today's date (even though it may not actually rotate the log - it uses that first run as a baseline if it's just interval-based).

Summary

For something that runs quietly in the background and only really performs one type of task, logrotate does quite a bit. You hopefully understand logrotate better than you wanted to. At the least you should be able to set up new logrotate config files for your own purposes, either creating them from scratch or copying existing configs and modifying them appropriately. And most importantly, you can keep your logs from getting out of control.